

Unifying communication paradigms in measurement-based delegated quantum computing

Fabian Wiesner^{1,2}, Jens Eisert^{2,3}, and Anna Pappa¹

¹ Electrical Engineering and Computer Science Department,
Technische Universität Berlin, 10587 Berlin, Germany

² Dahlem Center for Complex Quantum Systems,
Freie Universität Berlin, 14195 Berlin, Germany

³ Fraunhofer Heinrich-Hertz Institute, 10587 Berlin, Germany

Abstract. Delegated quantum computing (DQC) allows clients with low quantum capabilities to outsource computations to a server hosting a quantum computer. This process is often envisioned within the measurement-based quantum computing framework, as it naturally facilitates blindness of inputs and computation. Hence, the overall process of setting up and conducting the computation encompasses a sequence of three stages: preparing the qubits, entangling the qubits to obtain the resource state, and measuring the qubits to run the computation. There are two primary approaches to distributing these stages between the client and the server that impose different constraints on cryptographic techniques and experimental implementations. In the prepare-and-send setting, the client prepares the qubits and sends them to the server, while in the receive-and-measure setting, the client receives the qubits from the server and measures them. Although these settings have been extensively studied independently, their interrelation and whether setting-dependent theoretical constraints are inevitable remain unclear. By implementing the key components of most DQC protocols in the respective missing setting, we provide a method to build prospective protocols in both settings simultaneously and to translate existing protocols from one setting into the other.

Keywords: Delegated Quantum Computing, Quantum verification, Composable Cryptography

1 Introduction

The potential for achieving unparalleled computational power beyond what is available with classical computers renders quantum computers highly appealing to academic institutions and companies alike. However, they are costly, prone to errors, and demand specialized maintenance and cooling systems beyond the capacity of standard laboratories. *Delegated quantum computing* (DQC) addresses these challenges, at least in principle, on a meaningful level of abstraction: it enables clients with limited quantum resources to outsource quantum computations to a high-performance quantum server.

Two crucial properties distinguish DQC from classical cloud access as it is already used today. *Blindness* guarantees that the server cannot learn the input to the computation or the computation itself. *Verifiability* allows the client to verify the output of the computation. The impossibility result for classical remote state preparation [1] suggests that a purely classical client is likely not able to achieve blind DQC without further assumptions in an information-theoretical composable fashion, but by leveraging computational assumptions as used in Refs. [26,9]. However, a client with minimal quantum capabilities can blindly delegate a quantum computation when the latter is instantiated in the *measurement-based quantum computing model* (MBQC) [35,2], achieving even information-theoretic composable security.

Two separate lines of research – *prepare-and-send* (PS) and *receive-and-measure* (RM) – emerged. Both follow the convention of MBQC to encode computations as sequences of measurement angles in the XY-plane for qubit measurements of the resource state⁴. Since the outcomes of qubit measurements of entangled states are inherently random, some outcomes introduce unintended Pauli operations on the qubits adjacent to the measured one. These Pauli operations can be systematically propagated forward in the measurement sequence by adapting the angles using the *flow function* associated with the resource state. This technique eliminates the necessity to apply quantum gates during the measurement stage by deferring them to the output phase.

Despite following the same convention, PS and RM differ significantly in the distribution of the steps of an MBQC computation among the server and the clients. In the context of delegated quantum computing, an MBQC computation is a sequence of four steps: (i) preparing single qubits, (ii) entangling these qubits to obtain the resource state using controlled Z (CZ) gates, (iii) conducting the measurement sequence and (iv) applying the corrections on the outcome. While the server will always perform step (ii) in both settings, the clients' behavior changes: in PS, the clients have to perform (i) and (iv) while in RM, the clients conduct (iii) – instead of (i) – and (iv).

These different distributions of responsibilities imply a difference in the techniques used to achieve blindness and verifiability in each setting. Beyond the implications for cryptographic techniques used for the protocol, the choice of setting is also

⁴ Note that the choice of the XY-plane is conventional, albeit arbitrary. Other choices of planes and entangling operations work as well [15].

consequential for the technological implementation. For example, in PS the clients need to be able to prepare single-qubit states while in RM they have to perform measurements. The chosen setting also imposes constraints on the operations on the server's side, and while photons are likely to serve for encoding flying qubits, it is uncertain which technology will enable the server to perform quantum computations. Indeed, some first proof-of-concept implementations of DQC in PS leverage photonic quantum computing [14,33], in contrast to a recent implementation in RM, which utilizes solid-state qubits [37]. To maintain flexibility regarding the hardware platform that will be used for future quantum servers, it is imperative to develop cryptographic techniques for both communication settings of DQC.

1.1 Related work

The first DQC protocol was proposed in 2005 by Childs in the circuit model [8] – however, most DQC protocols today build on notions of MBQC. Throughout the history of MBQC-based DQC, the PS and RM settings have been extensively, albeit separately, studied. The first protocol proposing delegated quantum computing using MBQC is the work of Broadbent et al. [4] in 2009. The PS protocol Broadbent et. al. as proposed achieves perfect blindness by using random offsets for the communicated measurement angles that cancel out offsets at the preparation – a technique that is now the standard approach for blindness in PS. While Ref. [4] has shown only stand-alone security of the protocol, Dunjko et al. [11] proved composable security in 2018. Many other works have built upon this first protocol. A protocol by Fitzsimons and Kashefi [12], proposed in 2017, is one of the first efforts toward verifiability. This protocol utilizes in PS that Z -eigenstates are invariant under CZ and, hence, prevent the establishment of entanglement. This mechanism allows splitting a resource state into sections, of which some serve for the actual computation and other parts, called traps, to test the honesty of the server. Crucially, the server does not know the partition of the resource state since the client sends the qubits for the resource state. Kashefi and Wallden [24] optimized this approach and proposed a protocol with significantly lower verification overhead, and Leichtle et al. [25] leveraged the properties of BQP computations to provide fault-tolerant verification of such computations, and Kapourniotis et al. provided an extensive analysis for several trapification patterns [20]. However, the number of clients was also varied: Kashefi and Pappa [23] proposed one of the first protocols allowing multiple clients to delegate a joint computation to an untrusted server, achieving blindness against the server or a subset of clients but not all coalitions of clients and the server. A recent work [21] in PS combines multi-client settings and verification – although not trap-based – and achieves blindness and verifiability as long as a single client is honest and the computation has a classical output utilizing a majority vote.

Parallel to these developments for PS, research in the RM setting produced several protocols, as well. The first is the perfectly blind single-client protocol by Morimae and Fujii in 2013 [32]. Shortly after that, Morimae proposed the first verification protocol in RM [30], in which – similarly to the scheme of Ref. [12] – the client introduces traps. However, since the client does not prepare the qubits, the traps are introduced by delegating a precomputation. A different approach to verification was used a year later in a protocol that utilizes stabilizer testing to verify the resource state with the caveat that the input of the computation can only be classical [16]. Building upon this, Morimae lifted this caveat in 2016 [31]. Interactive proofs for verifying quantum learning and testing have also been considered, leading to a variety of interactive proof protocols for specific learning and testing tasks, which allow memory-constrained quantum verifiers to gain significant advantages by delegating to untrusted provers [7].

Finally it is worth noting that all previously mentioned works, both in RM and PS, put trust in the clients' devices. In 2018, Hayashi and Hajdušek [17] replaced the assumption of fully trusted measurements in RM with the weaker assumption that the measurement device has no internal memory. This allowed them to use techniques from self-testing to verify the resource state and measurement device simultaneously. While this is an interesting approach to DQC, this type of untrusted client setup is outside the scope of our work.

1.2 Our contribution

While two protocols from different settings can exhibit similarities at a higher level by achieving or even utilizing the same techniques, the two settings are not obviously equivalent. More specifically, the interrelation of the settings is unknown, which raises the question of whether everything implemented in one setting can be implemented in the other with the same level of security. In our work, we follow a modular approach using the *abstract cryptography framework* [28] and answer this question positively on a practical level by investigating the abstract building blocks used in modern DQC protocols. We highlight existing or immediate correspondences between implementations of these building blocks, identify gaps in implementing these building blocks in the two settings, and close these gaps by providing the respective missing implementations. It is, however, important to note that we only consider implementations where the clients' devices are trusted in both settings, i.e., we assume trusted measurements in RM and trusted preparations in PS.

In more detail, we introduce a direct usage of traps in RM by means of proposing a trap-based equivalent of the trap-based verification protocol in Ref. [24] which is in PS, we translate verification based on stabilizer testing as used in Ref. [16], which seems native to RM, to PS, and finally, implement collective remote state preparation as proposed in Ref. [23] in RM.

Closing these gaps enables the setting-agnostic development of protocols, which ultimately provides more flexibility in both theoretical constructions and experimental implementations of such constructions.

Functionality/Implementation type	RM version	PS version
Single client blind DQC	[32]	[4]
Trap-based verification*	[30]	[12]
Optimized trap-based verification	This work	[24]
Stabilizer-based verification [†]	[16]	This work
Collective Remote State preparation	This work	[23]

Table 1. Overview of related works and the gaps this work closes.

*Note that [30] requires a precomputation to isolate the traps; this work also demonstrates that this is not required in general for trap-based verification in RM. Further, the resource states [30] and [12] have different structures, although the verification mechanism follows the similar ideas.

[†]The technique we provide can be used for several protocols. In Appendix B, we use it to provide a PS version of Ref. [16] as an example and provide a more general reduction to the RM setting if the server is dishonest.

We first introduce the abstract cryptography framework in section 2. In section 3, we present our results, i.e., the missing implementations of the building blocks of modern DQC. In the first subsection (3.1), we present and prove the security of an RM version of Ref. [24], followed by the translation of verification by stabilizer testing to PS. In the second subsection (3.2), we introduce collective remote state preparation in RM. Finally, we conclude our work in section 4.

2 Abstract cryptography

In order to demonstrate the modularity of the various protocols, we will use a composable security framework, namely *abstract cryptography* (AC) [28] which is closely related to constructive cryptography [27] and follows similar ideas as *universal composability* (UC) [6] and *categorical composable cryptography* (CCC) [5]. While we do not formally introduce every component of AC and refer to Refs. [28] and [34], we provide a brief explanation of how one defines security using these components.

The goal of a protocol in AC is to construct an ideal (target) resource from a real one. The AC framework utilizes distinguishability as measure of distance of the construction and the ideal resource which yields the security parameter. When we prove security, we aim to prove an upper bound for this distance – hence, assuming that there is an honest subset $\mathcal{H}$ of all parties $\mathcal{I}$, we need to show that the composition of the honest parties’ protocols attached to the real resource is indistinguishable up to some ϵ from the ideal resource, where a simulator acts on the interfaces of the dishonest parties.

Definition 1 (Secure construction). Let $\mathbf{R}_\# = (\mathbf{R}, \#)$ and $\mathbf{S}_b = (\mathbf{S}, b)$ be two pairs, each consisting of a resource with interface set $\mathcal{I}$ and a filter. For a set of honest parties $\mathcal{H}$, a protocol $\pi = \{\pi_i\}_{i \in \mathcal{I}}$ securely constructs $\mathbf{S}_b$ out of $\mathbf{R}_\#$ within ϵ , if there exists a simulator $\sigma_{\mathcal{I} \setminus \mathcal{H}}$ such that

$$(\pi_{\mathcal{H}} \circ \#_{\mathcal{H}}) \mathbf{R} \approx_{\epsilon} b_{\mathcal{H}} \mathbf{S} \sigma_{\mathcal{I} \setminus \mathcal{H}}. \quad (1)$$

In this case, we write $\mathbf{R}_\# \xrightarrow[\mathcal{H}]{\pi, \epsilon} \mathbf{S}_b$.

It is easier to understand the above definition with an example; in Fig. 1 we show how Eq. (1) looks like when $\mathcal{I} = \{A, B, C, D\}$ and $\mathcal{H} = \{A, B\}$. Note that we allow for global simulators in contrast to the usual local simulators in AC. This is not an unusual assumption and has been used before, both in AC [10], as well as in other frameworks ([6,5]).

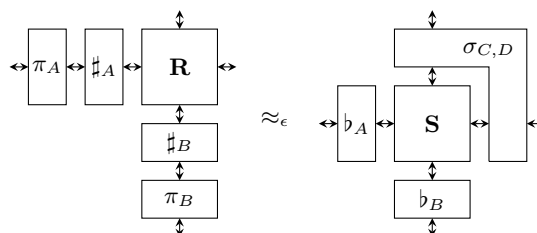


Fig. 1. Visualization of the secure construction with $\mathcal{I} = \{A, B, C, D\}$ and $\mathcal{H} = \{A, B\}$.

One shows security by proving the above definition for all sets of honest parties $\mathcal{H} \subseteq \mathcal{I}$ that are relevant for the security of the desired functionality. For example in QKD, it does not make sense to consider any of the two communicating parties to be dishonest; we are interested in proving a) correctness, where both the communicating parties and the eavesdropper are honest, and b) security, where the honest set contains only the communicating parties.

There are two reasons that motivate the definition of secure construction; first, any successful attack on the implementation implies an attack on the ideal resource, which one finds by composing the attack with the simulator. Hence, we can encode in the ideal resource what should be possible for dishonest parties. As the access to the ideal resource is different depending on whether a party is honest or not, we apply filters for honest parties that block such additional interactions.

The second motivation is *composability*: as distinguishability is a pseudo-metric which is non-increasing under composition, we find for any filtered resources $\mathbf{R}_\#$, $\mathbf{S}_b$ and $\mathbf{Q}_\#$ that

$$\mathbf{R}_\# \xrightarrow[\mathcal{H}]{\pi, \epsilon_1} \mathbf{S}_b \wedge \mathbf{S}_b \xrightarrow[\mathcal{H}]{\tau, \epsilon_2} \mathbf{Q}_\# \Rightarrow \mathbf{R}_\# \xrightarrow[\mathcal{H}]{\tau \circ \pi, \epsilon_1 + \epsilon_2} \mathbf{Q}_\#. \quad (2)$$

3 Implementing the components for DQC

Essentially, all proposed MBQC-based DQC protocols include implementations of at least some of the following three components and leverage their properties:

1. Single-client blind DQC ($\mathbf{S}^{\text{blind}}$).
2. Single-client verifiable DQC ($\mathbf{S}^{\text{ver}}$).
3. Collective *remote state preparation* (RSP).

Interestingly, not all these components have been implemented in both the PS and the RM setting. In this work, we provide the missing implementations and therefore enable the execution of any type of protocol utilizing these components in either of the two settings.

The first component, $\mathbf{S}^{\text{blind}}$ (cf. Fig. 2), has already been perfectly implemented in both PS [4] and RM [32] as long as the client is honest.

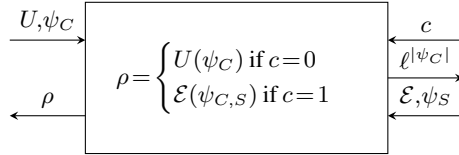


Fig. 2. Visualization of $\mathbf{S}^{\text{blind}}$. $\ell^{|\psi_C|}$ is the size of the register for the client's input ψ_C (which they might have obtained from some third party), $\mathcal{E}$ is a completely positive trace-preserving (CPTP) map to the space of linear operators on $\mathbb{C}^{\ell^{|\psi_C|}}$, ψ_S is a register of the server and c denotes the server's behavior. If the server is honest, we assume a filter $\#_S$ which inputs $c=0$, ignores the received dimensionality of the client's state and inputs any CPTP map and register.

In the following sections, we will therefore examine the two remaining components.

3.1 Single-client verifiable DQC

For the second component, $\mathbf{S}^{\text{ver}}$ (cf. Fig. 3), three types of implementations are used: the first is based on the cut-and-choose technique applied on $\mathbf{S}^{\text{blind}}$ [22], i.e., repeated runs with trap rounds, the second type of implementation uses separated traps in the resource state [12,24] and the third leverages stabilizer measurements on the resource state [16].

The first type of implementation of $\mathbf{S}^{\text{ver}}$ uses a composition of many instances of $\mathbf{S}^{\text{blind}}$, where some of them are used as traps to check the behavior of the server. For protocols that use test rounds such as Ref. [3], one can treat $\mathbf{S}^{\text{blind}}$ as a black box to build $\mathbf{S}^{\text{ver}}$, and therefore the equivalence of the $\mathbf{S}^{\text{blind}}$ implementations between the PS and the RM settings implies the equivalence of the cut-and-choose implementations of $\mathbf{S}^{\text{ver}}$.

For the second type, we present in Protocol 1 an RM version of the optimized trap-based verification protocol in Ref. [24]. We note that dummy nodes allow us to perform break operations, i.e., measuring a dummy deletes an edge. In this way, the traps are isolated, and the computational nodes are brought into the state of the dotted base state. The computation can be adapted so that one applies the bridge operation on the dots; this, together with the observation that for an honest server the isolated traps will be in $Z^{N_{DT(G)}(t)}|+\rangle$ and, hence, will not trigger abort, gives correctness.

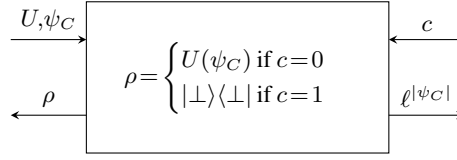


Fig. 3. Visualization of S^{ver} . $l^{|\psi_C|}$ is the size of the register for the client's input ψ_C (which they might have obtained from some third party). If the server is honest, we assume a filter $\mathfrak{b}_S$ which inputs $c=0$ and ignores the received dimensionality of the client's state. The actual verification property is encoded into the behavior of the resource if $c=1$. Since $|\perp\rangle\langle\perp|$ is orthogonal to the space of possible honest outputs, the client can easily learn whether the server cheated or not.

OUR CONTRIBUTION (TRAP-BASED VERIFICATION):

In Protocol 1, the trace of the output projected into the subspace that is orthogonal to the honest output is upper bounded by $8/9$ if the client accepted the computation. I.e., Protocol 1 is $8/9$ -stand alone verifiable.

Protocol 1 RM version of optimized trap-based verification [24]

Input: Client inputs a quantum input ψ_C along with $|+\rangle\langle+|^{\otimes 2l^{|\psi_C|}}$ of unspecified origin and secret measurement angles encoding U .

Output: $U(\psi_C)$ if the server is honest, $|\perp\rangle\langle\perp|$ otherwise.

- 1: Similar to the optimized trap-based verification [24], the client one-time pads the input state ψ_C and the additional register $|+\rangle\langle+|^{\otimes 2l^{|\psi_C|}}$. The combined state is denoted with $|e\rangle\langle e|$.
 - 2: The server prepares the resource state according to the dotted triple-graph and entangles it with $|e\rangle\langle e|$, the resulting state is called ρ .
 - 3: The client samples a coloring respecting the position of the input in $|e\rangle\langle e|$, which partitions the nodes of $DT(G)$ into computational nodes C , dummy nodes D and trap nodes T .
 - 4: The server sends ρ node by node to the client.
 - 5: The client measures each dummy d in the Z basis, and notes the correction Z^{r_d} for the neighbors, where r_d is the measurement outcome.
 - 6: The client measures the nodes in C to perform the computation while respecting the corrections introduced by the outcomes of the previous measurements and the offsets for the input.
 - 7: The client measures the trap nodes T in the X basis and aborts if there is a node t such that the measurement outcome r_t does not fulfill $r_t = r_{N_{DT(G)}(t)} := \bigoplus_{d \in N_{DT(G)}(t)} r_d$, where $N_{DT(G)}(t)$ denotes the neighborhood of t .
 - 8: If the client does not abort, it corrects the outcome of the computation before outputting it.
-

Similar to previous works on trap-based verification in PS [24, 12] and RM [30], we start by showing stand-alone security. In the proof (given in the Appendix), we show an upper bound for the probability of accepting an output that is in a subspace orthogonal to the honest outcomes. Composable security follows directly from the reduction to local criteria as shown in Ref. [11]. However, this reduction creates an overhead, and in order to obtain an implementation that is close to the ideal resource, the stand-alone security would need to be high. To achieve this, one could, for example, use a majority vote on the classical outputs of multiple instances of the protocol or fault-tolerant codes such as the RHG code [36] for the computation. A relevant result uses the majority vote for noisy BQP computations [25]. Notably, this protocol does not require enlarging the resource state to include traps for BQP computations; since the input and output are both classical, one can consider pure computation rounds without traps and use the other rounds for traps. Furthermore, the security proof does not rely on the protocol being in the prepare-and-send setting. Hence, the analysis directly encompasses the receive-and-measure variant in which the client measures in the Z -basis for the dummy nodes in trap rounds as in Protocol 1.

The third type of implementation of S^{ver} utilizes stabilizer measurements on the resource state. In the RM setting, stabilizer measurements allow the client to verify the state that the server sent [16], leveraging the fact that the resource state is a graph state [19, 18] and hence a stabilizer state.

OUR CONTRIBUTION (VERIFICATION BY STABILIZER TESTING):

For every stabilizer measurement of the graph state that the server prepared in RM, we provide a preparation procedure for the client in PS that allows them to blindly delegate an equivalent stabilizer measurement to the server. The procedure in PS provides the same composable security and the one in RM, which is shown with a reduction from PS to RM with $\varepsilon=0$.

Generators of the stabilizer set of a graph state for a graph $G=(V, E)$ are given by

$$g_j = X_j \otimes \left(\bigotimes_{i \in N_G(j)} Z_i \right) \otimes \left(\bigotimes_{i \in V \setminus (N_G(j) \cup \{j\})} \mathbb{1}_i \right) \quad (3)$$

for all $j \in V$. To translate this mechanism in the PS setting, the server would need to perform measurements on the graph state; however, Z measurements cannot be blindly delegated as they cannot be hidden with angle offsets. To derive an equivalent condition for verification, we denote $g = \{\sigma_n\}_{n \in V}$ be a randomly chosen stabilizer of the resource state with $\sigma_n \in \{\mathbb{1}, X, Y, Z\}$, i.e.,

$$\left(\bigotimes_{n \in V} \sigma_n \right) \mathcal{E}_G \left(\bigotimes_{n \in V} |+_n\rangle \right) = \mathcal{E}_G \left(\bigotimes_{n \in V} |+_n\rangle \right), \quad (4)$$

where $\mathcal{E}_G$ is a product on CZ gates, one for each edge in G . It holds that the above implies that the outcomes for the non-trivial nodes add up to an even number when one measures the state with a stabilizer measurement corresponding to g . Hence, with V_g being the set of vertices for which $\sigma_n \neq \mathbb{1}$, $\bar{V}_g = V \setminus V_g$, and for $\mathbf{r} \in \{0,1\}^{|V_g|}$ we denote $|\mathbf{r}| = \sum_{n \in V_g} r_n$, we find

$$1 = \sum_{\substack{\mathbf{r} \in \{0,1\}^{|V_g|} \\ |\mathbf{r}| \text{ even}}} \text{Tr} \left\{ \left[\left(\bigotimes_{n \in V_g} |\sigma_n(r_n)\rangle \langle \sigma_n(r_n)| \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \mathbb{1}_n \right) \right] \mathcal{E}_G \left(\bigotimes_{n \in V} |+_n\rangle \langle +_n| \right) \mathcal{E}_G \right\}, \quad (5)$$

where $|\sigma_n(r_n)\rangle \langle \sigma_n(r_n)|$ denotes the projector onto the $(-1)^{r_n}$ eigenspace of σ_n and we used $\mathcal{E}_G = \mathcal{E}_G^\dagger$. Now, let $\zeta(n) = 0$ if $\sigma_n = Z$ and 1 otherwise. Using that X flips the outcome of Z measurement and Z flips the outcomes of X and Y measurements, we find equivalently

$$0 = \sum_{\substack{\mathbf{r} \in \{0,1\}^{|V_g|} \\ |\mathbf{r}| \text{ odd}}} \text{Tr} \left\{ \left[\left(\bigotimes_{n \in V_g} Z_n^{r_n \zeta(n)} X_n^{r_n(\zeta(n) \oplus 1)} |\sigma_n(0)\rangle \langle \sigma_n(0)| X_n^{r_n(\zeta(n) \oplus 1)} Z_n^{r_n \zeta(n)} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \mathbb{1}_n \right) \right] \right. \\ \left. \mathcal{E}_G \left(\bigotimes_{n \in V} |+_n\rangle \langle +_n| \right) \mathcal{E}_G \right\}. \quad (6)$$

From now on, we denote $|\eta_n\rangle \langle \eta_n| = X_n^{r_n(\zeta(n) \oplus 1)} |\sigma_n(0)\rangle \langle \sigma_n(0)| X_n^{r_n(\zeta(n) \oplus 1)}$. Using the cyclicity of the trace and the commutation of Z and CZ , one finds that for any fixed $\mathbf{r}$ each term in the sum from above is equal to

$$\text{Tr} \left\{ \left[\left(\bigotimes_{n \in V_g} Z_n^{r_n \zeta(n)} |\eta_n\rangle \langle \eta_n| Z_n^{r_n \zeta(n)} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \mathbb{1}_n \right) \right] \mathcal{E}_G \left[\left(\bigotimes_{n \in V \setminus \{w\}} |+_n\rangle \langle +_n| \right) \otimes Z_w |+_w\rangle \langle +_w| Z_w \right] \mathcal{E}_G \right\} \quad (7)$$

for any $w \in \bar{V}_g$. Hence, we find

$$0 = \frac{1}{2^{|\bar{V}_g|}} \sum_{\mathbf{s} \in \{0,1\}^{|\bar{V}_g|}} \sum_{\substack{\mathbf{r} \in \{0,1\}^{|V_g|} \\ |\mathbf{r}| \text{ odd}}} \text{Tr} \left\{ \left[\left(\bigotimes_{n \in V_g} Z_n^{r_n \zeta(n)} |\eta_n\rangle \langle \eta_n| Z_n^{r_n \zeta(n)} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \mathbb{1}_n \right) \right] \right. \\ \left. \mathcal{E}_G \left[\left(\bigotimes_{n \in V_g} |+_n\rangle \langle +_n| \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} Z_n^{s_n} |+_n\rangle \langle +_n| Z_n^{s_n} \right) \right] \mathcal{E}_G \right\}. \quad (8)$$

Using again the cyclicity of the trace, the previously mentioned commutation relation, and algebraic simplifications, this implies after a few steps

$$\begin{aligned}
0 &= \sum_{\mathbf{s} \in \{0,1\}^{|\bar{V}_g|}} \sum_{\substack{\mathbf{r} \in \{0,1\}^{|\bar{V}_g|} \\ |\mathbf{r}| \text{ odd}}} \text{Tr} \left\{ \left[\left(\bigotimes_{n \in V_g} |\eta_n\rangle\langle\eta_n| \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \frac{\mathbb{1}_n}{2} \right) \right] \right. \\
&\quad \left. \mathcal{E}_G \left[\left(\bigotimes_{n \in V_g} Z_n^{r_n \zeta(n)} |+_n\rangle\langle+_n| Z_n^{r_n \zeta(n)} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} Z_n^{s_n} |+_n\rangle\langle+_n| Z_n^{s_n} \right) \right] \mathcal{E}_G \right\} \\
&= \sum_{\mathbf{s} \in \{0,1\}^{|\bar{V}_g|}} \sum_{\substack{\mathbf{r} \in \{0,1\}^{|\bar{V}_g|} \\ |\mathbf{r}| \text{ odd}}} \text{Tr} \left\{ \left[\left(\bigotimes_{n \in V_g} Z_n^{r_n \zeta(n)} |+_n\rangle\langle+_n| Z_n^{r_n \zeta(n)} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} Z_n^{s_n} |+_n\rangle\langle+_n| Z_n^{s_n} \right) \right] \right. \\
&\quad \left. \mathcal{E}_G \left[\left(\bigotimes_{n \in V_g} |\eta_n\rangle\langle\eta_n| \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \frac{\mathbb{1}_n}{2} \right) \right] \mathcal{E}_G \right\} \\
&= \sum_{\substack{\mathbf{r} \in \{0,1\}^{|\bar{V}_g|} \\ |\mathbf{r}| \text{ odd}}} \text{Tr} \left\{ \left(\bigotimes_{n \in V_g} Z_n^{r_n \zeta(n)} |+_n\rangle\langle+_n| Z_n^{r_n \zeta(n)} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \mathbb{1}_n \right) \mathcal{E}_G \left(\bigotimes_{n \in V_g} |\eta_n\rangle\langle\eta_n| \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \frac{\mathbb{1}_n}{2} \right) \mathcal{E}_G \right\}.
\end{aligned} \tag{9}$$

Finally, we can introduce a random Z flip for the nodes in the Z eigenbasis, since that does not change the value of the expression. Using the commutation relations, the cyclicity, and defining $V_Z \subseteq V_g$ as the set of nodes for which the stabilizer element is Z , we find

$$\begin{aligned}
0 &= \sum_{\mathbf{b} \in \{0,1\}^{|\bar{V}_Z|}} \sum_{\substack{\mathbf{r} \in \{0,1\}^{|\bar{V}_g|} \\ |\mathbf{r}| \text{ odd}}} \text{Tr} \left\{ \left[\left(\bigotimes_{n \in V_g} Z_n^{r_n \zeta(n)} |+_n\rangle\langle+_n| Z_n^{r_n \zeta(n)} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \mathbb{1}_n \right) \right] \right. \\
&\quad \left. \mathcal{E}_G \left[\left(\bigotimes_{n \in V_g \setminus V_Z} |\sigma_n(0)\rangle\langle\sigma_n(0)| \right) \otimes \left(\bigotimes_{n \in V_Z} Z^{b_n} X^{r_n(\zeta(n) \oplus 1)} |\sigma_n(0)\rangle\langle\sigma_n(0)| X^{r_n(\zeta(n) \oplus 1)} Z^{b_n} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \frac{\mathbb{1}_n}{2} \right) \right] \mathcal{E}_G \right\} \\
&= \sum_{\substack{\mathbf{r} \in \{0,1\}^{|\bar{V}_g|} \\ |\mathbf{r}| \text{ odd}}} \text{Tr} \left\{ \left[\left(\bigotimes_{n \in V_g \setminus V_Z} Z_n^{r_n \zeta(n)} |+_n\rangle\langle+_n| Z_n^{r_n \zeta(n)} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g \cup V_Z} \mathbb{1}_n \right) \right] \right. \\
&\quad \left. \mathcal{E}_G \left[\left(\bigotimes_{n \in V_g} X^{r_n(\zeta(n) \oplus 1)} |\sigma_n(0)\rangle\langle\sigma_n(0)| X^{r_n(\zeta(n) \oplus 1)} \right) \otimes \left(\bigotimes_{n \in \bar{V}_g} \frac{\mathbb{1}_n}{2} \right) \right] \mathcal{E}_G \right\},
\end{aligned} \tag{10}$$

where the introduced Z operators have been moved to the measurement part, such that the set of nodes with irrelevant outcomes became $\bar{V}_g \cup V_Z$ (which is $(V \setminus V_g) \cup V_Z$) and plugged in the definition of $|\eta_n\rangle\langle\eta_n|$.

This equation means operationally, that for any stabilizer g of the graph state corresponding to the graph G , the client prepares the nodes in the $+1$ -eigenstate if the stabilizer element is X or Y , in $|r_n\rangle$ for $r_n \in \{0,1\}$ if the stabilizer element is Z , and in the maximally mixed state if it is $\mathbb{1}$. Then the client delegates X measurements blindly to the server. If the results for the X and Y nodes and the preparation choices for the Z nodes add up to an odd number, the client rejects; otherwise, the server passes the test.

The above procedure links trap-based verification and stabilizer testing; more specifically, the dummies in trap-based verification are the Z measurements in stabilizer testing. However, in trap-based verification, the graph state is often ‘split up’ by Z preparations. This partitioning of the graph state could be viewed as measuring partial stabilizers after splitting the state via Z measurements. In Appendix B, we present in more detail an example based on Ref. [16] for which we propose and prove the security of a PS version. Further, we analyse the performance of general PS tests and find that the result in the same security their counter parts in RM.

3.2 Collective remote state preparation

The third component is collective remote state preparation, which is implemented in the PS setting in Refs. [23,21]. This is a resource that involves n clients and a server. Here we implement a corresponding resource in RM. To do so, we first introduce the ideal resource, which is slightly different than the one in Ref. [21], in that now all clients have some access to the resource⁵.

If all clients are honest (denoted with bit $c_j = 0$), a client C_k is able to prepare a state vector $|+\theta\rangle$ at the outer interface of the server, where θ remains secret. If however, some of the clients are dishonest (denoted by $c_j = 1$), they can input states ρ_j . One of these states (e.g., the state from the client C_ℓ with the highest identifier) is used for the output so that the state $Z^\theta(\rho_\ell)$ is outputted to the server instead of the state vector $|+\theta\rangle$.

Resource 1 The ideal resource **RSP** for Remote state preparation (RSP)

Input: C_k inputs θ . All other clients C_j input $c_j \in \{0,1\}$

and a qubit register ρ_j .

Output: ρ_S to the server.

```

1: if  $\forall j, j \neq k : c_j = 0$  then
2:    $\rho_S = |+\theta\rangle\langle +\theta|$ 
3: else
4:    $\ell = \max\{j | c_j = 1\}$ 
5:    $\rho_S = Z^\theta(\rho_\ell)$ 
6: end if

```

For each client $j \neq k$, we also consider the filter $\mathbb{I}_i$ that inputs $c_j = 0$ and the state $|0\rangle\langle 0|$. We consider the following protocol:

Protocol 2 The protocol π^{RSP} implements RSP in RM from a collection **QC** of quantum and classical channels.

Input: Client k inputs θ .

Output: The server outputs a qubit register ψ_S .

```

1: The server prepares the register  $\psi_S$  in  $|+\rangle\langle +|$  and for each client  $C_j$  a register  $\psi_j$  in  $|0\rangle\langle 0|$ .
2: The server applies for each client  $C_j$  with  $\psi_S$  as control and  $\psi_j$  as target register.
3: The server sends  $\psi_j$  to corresponding client  $C_j$ .
4: for each client  $C_j$  do
5:    $C_j$  samples  $\theta_j \leftarrow_{\$} \mathcal{A}$  with  $\mathcal{A} = \{\ell\pi/8 \mid 0 \leq \ell < 8\}$ 
6:    $C_j$  measures the received register in
      $\{|+\theta_j\rangle\langle +\theta_j|, |-\theta_j\rangle\langle -\theta_j|\}$ , the result is  $r_j$ .
7:   if  $j \neq k$  then
8:      $C_j$  sends  $(\theta_j, r_j)$  to  $C_k$ .
9:   else
10:     $C_k$  receives  $(\theta_j, r_j)$  from all clients.
11:     $C_k$  samples  $b \leftarrow_{\$} \{0,1\}$  and computes
        $\delta = (-1)^b \theta - \sum_{i=1}^n \theta_i - \pi \bigoplus_{i=1}^n r_i$ .
12:     $C_k$  sends  $b, \delta$  to the server.
13:   end if
14: end for
15: The server receives the correction  $\delta$  from the client  $C_k$ , applies  $X^b Z^\delta$  on  $\psi_S$  and outputs the register  $\psi_S$ .

```

We find correctness by observing that, for each register ψ_j of the clients, $Z^{\theta_j + r_j \pi}$ is applied on ψ_S . Hence, the final correction that the server applies brings the register in the state $|+\theta\rangle\langle +\theta|$. We now assume that the server is honest and some clients are dishonest. We define with $\mathcal{D}$ the set of dishonest and $\mathcal{H}$ the set of honest clients. As the server first entangles the registers, the state the distinguisher gets in the above implementation is

$$\rho_{\text{impl}} = \left(\left(\bigotimes_{j \in \mathcal{H}} \langle 0_j | + e^{i(\theta_j + r_j \pi)} | 1_j \rangle \right) \otimes \left(\bigotimes_{j \in \mathcal{D}} \mathbb{I}_j \right) \otimes X_S^b Z_S^{(-1)^b \theta - \theta'} \right) \left(\bigotimes_{j=1}^n C X_{S,j} \right) \left(\left(\bigotimes_{j \in \mathcal{H}} | 0_j \rangle \right) \otimes \left(\bigotimes_{j \in \mathcal{D}} | 0_j \rangle \right) \otimes | +_S \rangle \right), \quad (11)$$

⁵ The reason why we needed to introduce a new ideal resource is that the one introduced in Ref. [21] cannot be implemented in the case of dishonest clients and honest server.

where $\theta' = \sum_{j=1}^n \theta_j + r_j \pi$. Using the functionality of the circuit, the fact that Z -rotations on the control register commute with CX and $X^b Z^{(-1)^b \theta} = Z^\theta X^b$ we find

$$\rho_{\text{impl}} = \left(\left(\bigotimes_{j \in \mathcal{D}} \mathbb{1}_j \right) \otimes Z_S^\theta X_S^b Z_S^{-\theta_{\mathcal{D}}} \right) \left(\bigotimes_{j \in \mathcal{D}} CX_{S,j} \right) \left(\left(\bigotimes_{j \in \mathcal{D}} |0_j\rangle \right) \otimes |+_S\rangle \right) \quad (12)$$

where $\theta_{\mathcal{D}} = \sum_{j \in \mathcal{D}} \theta_j + r_j \pi$. The simulator $\sigma_{\mathcal{D}}$ (given in Simulator 1) emulates the protocol of the server and the classical part of the protocol of C_k with input $\theta = 0$. It inputs the state it gets by emulating the server to the interface of the dishonest client with the highest identifier.

Simulator 1 $\sigma_{\mathcal{D}}$ is the simulator if only a set of clients $\mathcal{D}$ is dishonest.

- 1: $\sigma_{\mathcal{D}}$ prepares the register ψ_S in $|+\rangle\langle+|$ and for each client in C_j with $j \in \mathcal{D}$ a register ψ_j in $|0\rangle\langle 0|$.
 - 2: $\sigma_{\mathcal{D}}$ applies for each client in $\mathcal{D}$, CX with ψ_S as control and ψ_j as target register.
 - 3: $\sigma_{\mathcal{D}}$ sends ψ_j to corresponding client C_j .
 - 4: $\sigma_{\mathcal{D}}$ receives (θ_j, r_j) from all clients in $\mathcal{D}$.
 - 5: $\sigma_{\mathcal{D}}$ samples $b \leftarrow_{\$} \{0,1\}$ and computes $\delta = -\sum_{i \in \mathcal{D}} \theta_i - \pi \bigoplus_{i \in \mathcal{D}} r_i$.
 - 6: $\sigma_{\mathcal{D}}$ applies $X^b Z^\delta$ on ψ_S .
 - 7: $\sigma_{\mathcal{D}}$ inputs $c_\ell = 1$ and ψ_S to the interface of client ℓ of **RSP** with $\ell = \max \mathcal{D}$. For the other clients in $\mathcal{D}$ it inputs $c_j = 0$.
-

Hence, we need to consider two rounds of corrections; the first is applied by the simulator with $\delta = -\theta_{\mathcal{D}}$ (since the simulator sets $\theta = 0$), and the second is applied by the ideal resource with Z_S^θ . We then find

$$\rho_{\text{sim}} = \left(\left(\bigotimes_{j \in \mathcal{D}} \mathbb{1}_j \right) \otimes Z_S^\theta \right) \left(\left(\bigotimes_{j \in \mathcal{D}} \mathbb{1}_j \right) \otimes X^b Z_S^{-\theta_{\mathcal{D}}} \right) \left(\bigotimes_{j \in \mathcal{D}} CX_{S,j} \right) \left(\left(\bigotimes_{j \in \mathcal{D}} |0_j\rangle \right) \otimes |+_S\rangle \right) = \rho_{\text{impl}}, \quad (13)$$

which implies perfect indistinguishability, i.e., $\pi_{\mathcal{H}}^{RSP} \mathbf{QC} \pi_S^{RSP} = \#_{\mathcal{H}} \mathbf{RSP} \sigma_{\mathcal{D}}$, if the server is honest.

Simulator 2 $\sigma_{\mathcal{D} \cup \{S\}}$ is the simulator if a set of clients $\mathcal{D}$ and the server are dishonest.

- 1: $\sigma_{\mathcal{D} \cup \{S\}}$ inputs $c_j = 0$ for every $j \in \mathcal{D}$.
 - 2: $\sigma_{\mathcal{D} \cup \{S\}}$ receives ψ_I from the ideal resource at the server's interface.
 - 3: $\sigma_{\mathcal{D} \cup \{S\}}$ receives for each client $j \in \mathcal{H}$ ψ_j from the server.
 - 4: **for** each client C_j in $\mathcal{H} \setminus \{k\}$ **do**
 - 5: $\sigma_{\mathcal{D} \cup \{S\}}$ samples $\theta_j \leftarrow_{\$} \mathcal{A}$.
 - 6: $\sigma_{\mathcal{D} \cup \{S\}}$ receives ψ_j from the server
 - 7: $\sigma_{\mathcal{D} \cup \{S\}}$ measures the received register in $\{|+\theta_j\rangle\langle+\theta_j|, |-\theta_j\rangle\langle-\theta_j|\}$, the result is r_j .
 - 8: **end for**
 - 9: $\sigma_{\mathcal{D} \cup \{S\}}$ applies $CX_{k,I}$ on ψ_k and ψ_I , measures ψ_I in $\{|0\rangle\langle 0|, |1\rangle\langle 1|\}$ and saves the outcome as b .
 - 10: $\sigma_{\mathcal{D} \cup \{S\}}$ samples $\theta_k \leftarrow_{\$} \mathcal{A}$, measures the register ψ_k in $\{|\pm\theta_k\rangle\langle\pm\theta_k|\}$ and saves the result as r_k .
 - 11: $\sigma_{\mathcal{D} \cup \{S\}}$ receives θ_i, r_i for $i \in \mathcal{D}$, computes $\delta = -\sum_{j=1}^n \theta_j + \pi \bigoplus_{j=1}^n r_j$ and outputs δ and $b \oplus 1$ to the server.
-

If the server is dishonest, we denote the register the simulator $\sigma_{\mathcal{D} \cup \{S\}}$ (given in Simulator 2) receives from the ideal resource as $\psi_I = |+\theta\rangle\langle+\theta|$. The simulator implements the protocol of all honest clients but C_k and inputs $c_j = 0$ for all dishonest clients. Instead of implementing the protocol of C_k , the simulator applies $CX_{k,I}$ on the register ψ_k it receives from the server and ψ_I . After that the simulator measures ψ_I in $\{|0\rangle\langle 0|, |1\rangle\langle 1|\}$ and saves the outcome as b . This combination of $CX_{k,I}$ followed by the measurement is equivalent to applying $Z_k^{(-1)^b \theta}$ on ψ_k and is the main mechanism exploited for the PS version of the implementation. At last, the simulator finds r_k by measuring the register ψ_k in $\{|\pm\theta_k\rangle\langle\pm\theta_k|\}$ with a random θ_k , computes

$$\delta = -\sum_{j=1}^n \theta_j + \pi \bigoplus_{j=1}^n r_j \quad (14)$$

and sends $\delta, b \oplus 1$ to the distinguisher at the server's interface. Note that for dishonest clients, the distinguisher and for honest clients, but client k , the implementation inside the simulator provides θ_j, r_j . We find the map

$$\frac{1}{2 \cdot 8^{|\mathcal{H}|}} \sum_{\substack{\theta \in \mathcal{A}^{|\mathcal{H}|} \\ \mathbf{r} \in \{0,1\}^{|\mathcal{H}|} \\ b \in \{0,1\}}} \text{Tr} \left\{ \left| +^{-\theta_k - (-1)^b \theta - r_k \pi} \right\rangle \left\langle +^{-\theta_k - (-1)^b \theta - r_k \pi} \right| \psi_k \right\} \left(\prod_{\substack{j \in \mathcal{H} \\ j \neq k}} \text{Tr} \left\{ \left| +^{-\theta_j - r_j \pi} \right\rangle \left\langle +^{-\theta_j - r_j \pi} \right| \psi_j \right\} \right) \quad (15)$$

$$\left| -\sum_{j=1}^n \theta_j - \pi \bigoplus_{j=1}^n r_j, b \oplus 1 \right\rangle \left\langle -\sum_{j=1}^n \theta_j - \pi \bigoplus_{j=1}^n r_j, b \oplus 1 \right|$$

in the simulation. As θ_k and b are uniformly distributed, we can first replace $\theta_k \mapsto \theta_k + (-1)^b \theta$ and second $b \mapsto b \oplus 1$, which gives us

$$\frac{1}{2 \cdot 8^{|\mathcal{H}|}} \sum_{\substack{\theta \in \mathcal{A}^{|\mathcal{H}|} \\ \mathbf{r} \in \{0,1\}^{|\mathcal{H}|} \\ b \in \{0,1\}}} \left(\prod_{j \in \mathcal{H}} \text{Tr} \left\{ \left| +^{-\theta_j - r_j \pi} \right\rangle \left\langle +^{-\theta_j - r_j \pi} \right| \psi_j \right\} \right) \left| (-1)^b \theta - \sum_{j=1}^n \theta_j - \pi \bigoplus_{j=1}^n r_j, b \right\rangle \left\langle (-1)^b \theta - \sum_{j=1}^n \theta_j - \pi \bigoplus_{j=1}^n r_j, b \right| \quad (16)$$

which is exactly the map the implementation performs on the registers obtained from the server, i.e., the distinguishability is also zero when the server is dishonest, i.e., $\pi_{\mathcal{H}}^{RSP} \mathbf{QC} = \#_{\mathcal{H}} \mathbf{RSP} \sigma_{\mathcal{D} \cup \{S\}}$.

OUR CONTRIBUTION (COLLECTIVE REMOTE STATE PREPARATION):

Protocol 2 perfectly implements Resource 1 in the abstract cryptography framework as long as at least one client is honest, i.e., for every coalition of some clients not including client k with or without the server, there is a simulator such that perfect indistinguishability ($\varepsilon = 0$) is achieved.

4 Conclusion

Delegated quantum computation is crucial for enabling clients with limited quantum resources to securely and efficiently outsource complex quantum tasks to more powerful quantum servers. Protocols implementing specific functionalities have been proposed in the two prevalent communication settings, *prepare-and-send* (PS) and *receive-and-measure* (RM), and essentially use three components. Amongst these, only the simpler one, namely single-client blind DQC, is known to be perfectly implemented in both settings. Three techniques were used to implement the second component, which is single-client verifiable DQC. The first verification technique, cut-and-choose, i.e., intertwining verifiable computations with the actual computation in multiple rounds of blind DQC, inherits its equivalence from single-client blind DQC. The second technique leverages partitioning the resource state into multiple segments, allowing the use of some segments for verification while others serve for the actual computation. While an implementation of this technique in RM was proposed in 2014 [30], the development of the PS implementations [12,24] imposed a gap between the two settings. We closed this gap by proposing an RM version of optimized trap-based verification [24]. In contrast to the protocol in Ref. [30], our implementation does not require delegating a precomputation to split up the resource state, but utilizes measurements in the Z -basis following the same intuition as the PS implementations that leverage preparation in this basis. Our approach can be easily adapted for other protocols using this verification technique, such as [25]. The last verification technique used for single-client verifiable DQC is stabilizer testing [30]. Although intuitively stabilizer testing requires the client to measure the resource, we translated this technique to PS by leveraging the blindness of the server and deriving an equivalence between stabilizer tests in the two settings. We finally give an RM version of the third component, the remote state preparation protocol from Ref. [23]. This implementation directly implies RM versions of protocols in which this component has been used as a subroutine, such as in the works [23] and [21].

We have, therefore, demonstrated that the two communication models are, in fact, interchangeable in delegated quantum computation; for any (present or future) protocol that is proposed in one setting and consists of (some of) these three main components, there exists an equivalent one in the other setting, achieving the same levels of security. Our work not only clarifies the connection between the two communication settings and provides new protocols for DQC, but additionally opens paths for further research. For example, it enables research on previously unexplored hybrid communication settings, where clients that belong to different communication settings (e.g., a preparing client and a receiving client), can collaborate in order to delegate a multiparty computation to a server [23]. Our work also inspires further research of protocols like those of Ref. [13], in which the client sends weak coherent pulses instead of single photons, and raises the question of whether the communication direction in this semi-classical setting is crucial for security. An interesting question also arises regarding the trust in the clients' devices; specifically, whether the known RM protocols in which the devices of the clients are not trusted (see, e.g., Refs. [17,29]) can be translated to the PS setting. Finally, and also in light of recent implementations and technological advancements, we hope to motivate and simplify the exploration of further protocols to achieve practical delegated quantum computing.

5 Acknowledgments

The authors acknowledge support from the BMFTR (QR.X and QR.N), the European Union (Quantum Internet Alliance), the DFG (Emmy Noether grant No. 418294583), the European Research Council (ERC AdG DebuqC), the Einstein Research Unit on Quantum Devices and Berlin Quantum.

References

1. Badertscher, C., Cojocaru, A., Colisson, L., Kashefi, E., Leichtle, D., Mantri, A., Wallden, P.: Security limitations of classical-client delegated quantum computing. In: *Advances in Cryptology – ASIACRYPT 2020* (2020). https://doi.org/10.1007/978-3-030-64834-3_23
2. Briegel, H.J., Browne, D.E., Dür, W., Raussendorf, R., Van den Nest, M.: Measurement-based quantum computation. *Nature Phys.* **5**, 19–26 (2009). <https://doi.org/10.1038/nphys1157>
3. Broadbent, A.: How to verify a quantum computation. *Th. Comp.* **14**(1), 1–37 (2018). <https://doi.org/10.4086/toc.2018.v014a011>, <http://dx.doi.org/10.4086/toc.2018.v014a011>
4. Broadbent, A., Fitzsimons, J., Kashefi, E.: Universal blind quantum computation. *50th Ann. IEEE Symp. Found. Comp. Sc.* **2009**, 517 (2009). <https://doi.org/10.1109/focs.2009.36>
5. Broadbent, A., Karvonen, M.: Categorical composable cryptography. In: *Lecture Notes in Computer Science*, pp. 161–183. Springer International Publishing (2022). https://doi.org/10.1007/978-3-030-99253-8_9
6. Canetti, R.: Universally composable security: a new paradigm for cryptographic protocols. In: *42nd Ann. IEEE Symp. Found. Comp. Sc.* pp. 136–145 (2001). <https://doi.org/10.1109/SFCS.2001.959888>
7. Caro, M.C., Eisert, J., Hinsche, M., Ioannou, M., Nietner, A., Sweke, R.: Interactive proofs for verifying (quantum) learning and testing (2024), <https://arxiv.org/abs/2410.23969>
8. Childs, A.M.: Secure assisted quantum computation. *Quant. Inf. Comp.* **5**, 456–466 (2005)
9. Cojocaru, A., Colisson, L., Kashefi, E., Wallden, P.: Qfactory: Classically-instructed remote secret qubits preparation. *Advances in Cryptology – ASIACRYPT 2019* p. 615–645 (2019). https://doi.org/10.1007/978-3-030-34578-5_22
10. Colisson, L., Markham, D., Yehia, R.: All graph state verification protocols are composablely secure (2024). <https://doi.org/10.48550/arXiv.2402.01445>
11. Dunjko, V., Fitzsimons, J.F., Portmann, C., Renner, R.: Composable security of delegated quantum computation. In: *Advances in Cryptology – ASIACRYPT 2014* (2014). https://doi.org/10.1007/978-3-662-45608-8_22
12. Fitzsimons, J.F., Kashefi, E.: Unconditionally verifiable blind quantum computation. *Phys. Rev. A* **96**, 012303 (2017). <https://doi.org/10.1103/physreva.96.012303>
13. Garnier, M., Leichtle, D., Music, L., Ollivier, H.: Composablely secure delegated quantum computation with weak coherent pulses (2025), <https://arxiv.org/abs/2503.08559>
14. Greganti, C., Roehsner, M.C., Barz, S., Morimae, T., Walther, P.: Demonstration of measurement-only blind quantum computing. *New J. Phys.* **18**, 013020 (2016). <https://doi.org/10.1088/1367-2630/18/1/013020>
15. Hamrit, N., Perdrix, S.: Reversibility in extended measurement-based quantum computation. In: Krivine, J., Stefani, J.B. (eds.) *Reversible Computation*. pp. 129–138. Springer International Publishing, Cham (2015)
16. Hayashi, M., Morimae, T.: Verifiable measurement-only blind quantum computing with stabilizer testing. *Phys. Rev. Lett.* **115**, 220502 (2015). <https://doi.org/10.1103/PhysRevLett.115.220502>
17. Hayashi, M., Hajdušek, M.: Self-guaranteed measurement-based quantum computation. *Phys. Rev. A* **97**, 052308 (2018). <https://doi.org/10.1103/PhysRevA.97.052308>, <https://link.aps.org/doi/10.1103/PhysRevA.97.052308>
18. Hein, M., Dür, W., Eisert, J., Raussendorf, R., Van den Nest, M., Briegel, H.J.: Entanglement in graph states and its applications. In: *Proc. Int. School Phys. ‘Enrico Fermi’*. pp. 115 – 218. IOS Press, Amsterdam (2005)
19. Hein, M., Eisert, J., Briegel, H.J.: Multiparty entanglement in graph states. *Phys. Rev. A* **69**, 062311 (2004). <https://doi.org/10.1103/PhysRevA.69.062311>
20. Kapourniotis, T., Kashefi, E., Leichtle, D., Music, L., Ollivier, H.: Unifying quantum verification and error-detection: theory and tools for optimisations. *Quantum Sc. Tech.* **9**(3), 035036 (2024). <https://doi.org/10.1088/2058-9565/ad466d>, <https://doi.org/10.1088/2058-9565/ad466d>
21. Kapourniotis, T., Kashefi, E., Leichtle, D., Music, L., Ollivier, H.: Asymmetric secure multi-party quantum computation with weak clients against dishonest majority. *Quant. Sc. Tech.* **10**(2), 025015 (2025). <https://doi.org/10.1088/2058-9565/adaf12>, <https://dx.doi.org/10.1088/2058-9565/adaf12>
22. Kashefi, E., Music, L., Wallden, P.: The quantum cut-and-choose technique and quantum two-party computation (2017), <https://arxiv.org/abs/1703.03754>
23. Kashefi, E., Pappa, A.: Multiparty delegated quantum computing. *Crypt.* **1**, 12 (2017). <https://doi.org/10.3390/cryptography1020012>, <https://www.mdpi.com/2410-387X/1/2/12>
24. Kashefi, E., Wallden, P.: Optimised resource construction for verifiable quantum computation. *J. Phys. A* **50**, 145306 (2017). <https://doi.org/10.1088/1751-8121/aa5dac>
25. Leichtle, D., Music, L., Kashefi, E., Ollivier, H.: Verifying bqp computations on noisy devices with minimal overhead. *PRX Quantum* **2**(4), 040302 (2021)
26. Mahadev, U.: Classical verification of quantum computations. *SIAM J. Comp.* **51**(4), 1172–1229 (2022). <https://doi.org/10.1137/20M1371828>, <https://doi.org/10.1137/20M1371828>
27. Maurer, U.: Constructive cryptography – a new paradigm for security definitions and proofs. In: *Theory of Security and Applications*. pp. 33–56. Springer Berlin Heidelberg, Berlin, Heidelberg (2012). https://doi.org/10.1007/978-3-642-27375-9_3
28. Maurer, U., Renner, R.: Abstract cryptography. In: *Innovations in Computer Science*. Tsinghua University Press (2011)

29. McKague, M.: Interactive proofs for BQP via self-tested graph states. *Theory of Computing* **12**(3), 1–42 (2016). <https://doi.org/10.4086/toc.2016.v012a003>, <https://theoryofcomputing.org/articles/v012a003>
30. Morimae, T.: Verification for measurement-only blind quantum computing. *Phys. Rev. A* **89**, 060302 (2014). <https://doi.org/10.1103/PhysRevA.89.060302>
31. Morimae, T.: Measurement-only verifiable blind quantum computing with quantum input verification. *Phys. Rev. A* **94**, 042301 (2016). <https://doi.org/10.1103/PhysRevA.94.042301>, <https://link.aps.org/doi/10.1103/PhysRevA.94.042301>
32. Morimae, T., Fujii, K.: Blind quantum computation protocol in which alice only makes measurements. *Phys. Rev. A* **87**, 050301 (2013). <https://doi.org/10.1103/physrev.87.050301>
33. Polacchi, B., Leichtle, D., Carvacho, G., Milani, G., Spagnolo, N., Kaplan, M., Kashefi, E., Sciarrino, F.: Experimental verifiable multi-client blind quantum computing on a qline architecture (2024), <https://arxiv.org/abs/2407.09310>
34. Portmann, C., Renner, R.: Security in quantum cryptography. *Rev. Mod. Phys.* **94**(2), 025008 (2022). <https://doi.org/10.1103/revmodphys.94.025008>
35. Raussendorf, R., Briegel, H.J.: A one-way quantum computer. *Phys. Rev. Lett.* **86**, 5188–5191 (2001). <https://doi.org/10.1103/PhysRevLett.86.5188>, <https://link.aps.org/doi/10.1103/PhysRevLett.86.5188>
36. Raussendorf, R., Harrington, J., Goyal, K.: A fault-tolerant one-way quantum computer. *Ann. Phys.* **321**(9), 2242–2270 (2006). <https://doi.org/10.1016/j.aop.2006.01.012>
37. Wei, Y.C., Stas, P.J., Suleymanzade, A., Baranes, G., Machado, F., Huan, Y.Q.H., Knaut, C.M., Ding, W.S., Merz, M., Knall, E.N., Yazlar, U., Sirotin, M., Wang, I.W., Machielse, B., Yelin, S.F., Borregaard, J., Park, H., Lončar, M., Lukin, M.D.: Universal distributed blind quantum computing with solid-state qubits. *Science* **388**(6746), 509–513 (2025). <https://doi.org/10.1126/science.adu6894>

A Security proof for trap-based verification

Here we examine the stand-alone security of Protocol 1. If the server is dishonest, we can assume that it also has a register S in addition to the received state $|e\rangle\langle e|$. Without loss of generality, we further assume that the server first prepares the qubits of the resource state and then applies a joint unitary Ω on all the registers in its possession. After that, it entangles the registers of the resource state and the client's input into the dotted triple-graph, using operation $\mathcal{E}$. The state at the end of the protocol is given by

$$B(\nu) = \text{Tr}_S \left(\sum_s |s\rangle\langle s| \psi_s^\nu C_{\nu_C, s} \mathcal{E} \Omega \left(|0\rangle\langle 0|^{\otimes |S|} \otimes \left(\bigotimes_{n \in C, T, D} |+_n\rangle\langle +_n| \right) \otimes |e\rangle\langle e| \right) \Omega^\dagger \mathcal{E}^\dagger C_{\nu_C, s}^\dagger |\psi_s^\nu\rangle\langle s| \right), \quad (17)$$

where s is the vector of measurement results of the client, ν denotes the coloring and the encryption parameters used to obtain $|e\rangle\langle e|$, ψ_s^ν represents the corresponding measurement bases and $C_{\nu_C, s}$ represents the correction on the output that the client applies at the end of the protocol.

Note that, depending on the measurement outcomes of the dummies adjacent to the trap, the client needs to adapt the measurement basis for the traps corresponding to both the input and the resource state. Hence, the client accepts only if the measurement outcome of all traps is 0. We denote with $P_\perp$ the projector into the subspace that is orthogonal to the honest output. We find

$$p_{\text{fail}} = \sum_\nu p(\nu) \text{Tr} \left(\left(P_\perp \otimes \left(\bigotimes_{t \in T} |0_t\rangle\langle 0_t| \right) \right) B(\nu) \right), \quad (18)$$

where $p(\nu)$ denotes the probability of choosing a specific ν . Performing the trace over the auxiliary system S of the server turns the unitary Ω into a CPTP map, which can be expressed using the Pauli operators σ_i with complex coefficients $\alpha_{k,i}$,

$$p_{\text{fail}} = \sum_{\nu, s, i, j, k} p(\nu) \alpha_{k,i} \alpha_{k,j}^* \cdot \text{Tr} \left[\left(P_\perp \otimes \left(\bigotimes_{t \in T} |0_t\rangle\langle 0_t| \right) \right) |s\rangle\langle s| \psi_s^\nu C_{\nu_C, s} \mathcal{E} \sigma_i \left(\bigotimes_{n \in C, T, D} |+_n\rangle\langle +_n| \otimes |e\rangle\langle e| \right) \sigma_j \mathcal{E} C_{\nu_C, s}^\dagger |\psi_s^\nu\rangle\langle s| \right]. \quad (19)$$

Terms in which σ_i and σ_j are tensor products of $\mathbb{1}$ and X cannot contribute to the sum, except when X acts on the input. We define E to be the subset of Pauli operators that have at least one Y or Z on any of the registers or X on the input. If the base graph was used in a fault-tolerant setting, the number of operators in $\{Y, Z\}$ (or X on the input) would need to be the number of errors tolerated by the error detection code. However, without fault-tolerance, a single operator can map the output in the orthogonal subspace.

Using the cyclicity of the trace, assuming the attack mapped the state into an orthogonal subspace and defining s' as the substring of measurement results of nodes in D and C , we find

$$p_{\text{fail}} \leq \sum_{\nu, s', k} \sum_{i, j \in E} p(\nu) \alpha_{k,i} \alpha_{k,j}^* \text{Tr} \left[\left(\bigotimes_{t \in T} |\psi_t^\nu\rangle\langle \psi_t^\nu| \otimes |\psi_{s'}^\nu\rangle\langle \psi_{s'}^\nu| \right) \mathcal{E} \sigma_i \left(\bigotimes_{n \in C, T, D} |+_n\rangle\langle +_n| \otimes |e\rangle\langle e| \right) \sigma_j^\dagger \mathcal{E}^\dagger \right], \quad (20)$$

where the random flips r_t for all the trap measurements and the random offset θ_t for the input traps are represented by ψ_t^ν . Note that we dropped $C_{\nu_C, s}$ as it only acts on the output nodes that are already projected and traced out.

Following the previous works of trap-based verification in PS [24, 12], we utilize blindness in the next step; no matter which channel the server applies, the client's registers appear totally mixed to the server when considering the sum over the outcomes. This implies

$$p_{\text{fail}} \leq \sum_{\nu, k} \sum_{i, j \in E} p(\nu) \alpha_{k, i} \alpha_{k, j}^* \text{Tr} \left[\left(\bigotimes_{t \in T} |\psi_t^\nu\rangle \langle \psi_t^\nu| \right) \sigma_i \left(\bigotimes_{t \in T} |\psi_t^\nu\rangle \langle \psi_t^\nu| \otimes \frac{\mathbb{1}}{\text{Tr}(\mathbb{1})} \right) \sigma_j^\dagger \right]. \quad (21)$$

If $\sigma_j \neq \sigma_i$, they either differ on the input registers or on the traps. In the first case, the trace vanishes as all Pauli operators except the identity have trace 0. In the second case, the trace vanishes, since

$$\sum_{r_t} \sum_{\theta_t} \frac{1}{16} \text{Tr}(\langle \psi_t^\nu | \sigma_{i|t} | \psi_t^\nu \rangle \langle \psi_t^\nu | \sigma_{j|t} | \psi_t^\nu \rangle) = 0, \quad (22)$$

where $\sigma_{i|t}$ and $\sigma_{j|t}$ are single-qubit Pauli operators on trap t . Note that if the trap does not correspond to the input, there is no dependency on θ_t . Hence, only terms with $\sigma_i = \sigma_j$ contribute to the sum, and, therefore, we find

$$p_{\text{fail}} \leq \sum_{k, \nu^T} \sum_{i \in E} p(\nu^T) |\alpha_{k, i}|^2 \prod_{t \in T} \sum_{r_t, \theta_t} \frac{1}{16} (\langle \psi_t^\nu | \sigma_{i|t} | \psi_t^\nu \rangle)^2, \quad (23)$$

where ν has been split up into its contributing terms (i.e., positioning of the traps ν^T , r_t , and θ_t). Except for differences in notation, this is equation (C.10) in the work that proposes the optimized trap-based verification protocol [24]. As also the number and types of non-trivial attacks (element in E) are bijectively related, we can refer to the proof in Ref. [24] from here on and find that $p_{\text{fail}} \leq 8/9$.

B Example and security analysis for stabilizer testing in PS

As an example of how stabilizer testing in PS can provide new protocols, we will propose and prove a PS version of Ref. [16], a protocol that utilizes stabilizer testing on two-colorable graph states in RM to verify the server's honesty in a delegated quantum computation with a classical input. An example of a two-colorable graph state with the two measurement patterns used in Ref. [16] is depicted in Figure 4.

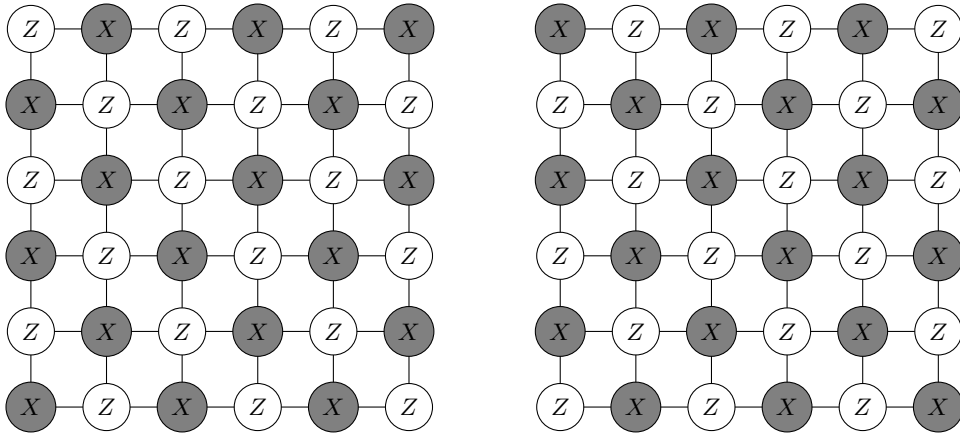


Fig. 4. The two measurement patterns on a two-colorable graph state. The figure is reproduced from Ref. [16].

The protocol proposed in Ref. [16] leverages the fact that one can measure on a two-colorable graph state half of the generators of the stabilizer set simultaneously. In order to perform this measurement, we associate one color (gray) with X measurements and the other (white) with Z measurements. Since canonical generators of the stabilizer are given by an X operator on a node n and Z operators on all of n 's neighbors, one can measure all stabilizers for which n is in a gray node. Alternatively, one can interchange the roles of X and Z to measure the complement of the previous set of generators in the set of all generators. This procedure gives two measurement patterns as depicted in Figure 4.

In the protocol proposed in Ref. [16], the server sends $2k+1$ copies of the resource state, which is a two-colorable graph state. The client measures a random subset of size k with one measurement pattern, another k copies randomly chosen from the

Protocol 3 π^{StabPS} implements stabilizer-based verification in PS as in Ref. [16]. The graph $G = (V, E)$ and its coloring $B \cup W = V$ are publicly known.

- 1: The client partitions the set $\{1, \dots, 2k+1\}$ randomly into T_1 , T_2 and C with $|T_1| = |T_2| = k$ and $|C| = 1$.
 - 2: The client sets $a \leftarrow \text{True}$
 - 3: **for** $i \in \{1, \dots, 2k+1\}$ **do**
 - 4: **for** $n \in V$ **do**
 - 5: If $i \in T_1 \wedge n \in B \vee i \in T_2 \wedge n \in W$ the client samples $r_n \leftarrow_{\$} \{0, 1\}$, prepares a qubit in $|r_n\rangle$ and sends it to the server.
 - 6: If $i \in T_2 \wedge n \in B \vee i \in T_1 \wedge n \in W$ the client samples $\theta_n \leftarrow_{\$} \{\ell\pi/8 \mid 1 \leq \ell < 8\}$, prepares a qubit in $|+\theta_n\rangle$ and sends it to the server.
 - 7: If $i \in C$ the client prepares the qubit for the computation rounds using the usual mechanisms for blindness as described in Ref. [4].
 - 8: **end for**
 - 9: The server entangles the received qubits according to the specifications of G .
 - 10: **if** $i \notin C$ **then**
 - 11: In the previous order for each node n : The client sends a random $\delta_n \leftarrow_{\$} \{\ell\pi/8 \mid 1 \leq \ell < 8\}$ if for n the qubit was prepared in the Z -eigenbasis and $\delta_n \leftarrow \theta_n + r'_n\pi$ otherwise, the server measures in $|\pm\delta_n\rangle\langle\pm\delta_n|$ and sends the result s_n to the client, who sets $r_n = r'_n \oplus s_n$ if the qubit for n was not prepared in the Z -eigenbasis.
 - 12: Now the client has for each $n \in V$ a bit r_n , either from the preparation or from the measurement step. If $\oplus_{n \in V} r_n = 1$ the client sets $a \leftarrow \text{False}$.
 - 13: **else**
 - 14: The server and the client act as required to conduct the computation (similar to [4]). The results labeled $o \in \{0, 1\}^d$.
 - 15: **end if**
 - 16: **end for**
 - 17: If $a = \text{True}$ the client outputs o , otherwise $\perp$.
-

remaining copies with the other measurement pattern, and uses the last one for the computation. Translating this procedure into PS as described in Section 3 gives the Protocol 3.

The correctness of this protocol follows directly from the analysis in Section 3 – the client will not abort (i.e., set the bit a to false) if the server is honest, since the offsets in the preparation and measurement cancel out each other. For security, we can follow the same steps as used in Ref. [11] to prove the security of UBQC. The client's quantum part of the protocol is equivalent to first preparing for each node an EPR pair, of which the client measures one half in the X basis if in the protocol they would have prepared a qubit in $|+\theta_n\rangle$ and denote the outcome as r'_n . If the node corresponds to a preparation in the Z -eigenbasis, the client measures the half in the Z -eigenbasis and denotes the result r_n . For each node n , the client applies Z^{θ_n} on the remaining half with a random θ_n before sending it to the server. Next, the client communicates for each $\delta_n = \theta_n + b_n\pi$ to the server and receives s'_n . The client computes $s_n = s'_n \oplus b_n$ and continues with s_n , r_n and r'_n as in the protocol.

Simulator 3 σ_S is the simulator for the server for stabilizer testing with any graph state based on $G = (V, E)$

- 1: **for** each node n in V **do**
 - 2: σ_S prepares $|EPR_{n,n'}\rangle = \frac{|0_n 0_{n'}\rangle + |1_n 1_{n'}\rangle}{\sqrt{2}}$
 - 3: σ_S samples $\theta_n \leftarrow_{\$} \mathcal{A}$ and applies $Z^{\theta_n}_{n'}$ on the second half labeled as n' .
 - 4: σ_S sends the second half to the server.
 - 5: **end for**
 - 6: **for** each node n in V **do**
 - 7: σ_S samples $b_n \leftarrow_{\$} \{0, 1\}$, computes $\delta_n = \theta_n + b_n\pi$ and sends δ_n to the server.
 - 8: σ_S receives s_n and computes $s_n = s'_n \oplus b_n$.
 - 9: σ_S sends the remaining half of $|EPR_{n,n'}\rangle$ and s_n to the client.
 - 10: **end for**
-

Since the preparation of the EPR pair, the application of Z^{θ_n} and the computation of $s_n = s'_n \oplus b_n$ do not depend on the type of the node or whether the round is used for computation or not, these steps can be outsourced to the simulator σ_S , given in Simulator 3. This basically leaves a protocol for the test rounds in which the client measures the stabilizer but learns and respects bit-flips that the server applied. We call this protocol $\pi^{HM15'}$. $\pi^{HM15'}$ is, in turn, equivalent to the one presented in Ref. [16] which we call $\pi^{HM15'}$, since another simulator σ'_S can introduce bit-flips or – the other way round (σ''_S) – store the state, wait for the messages, and cancel the bit-flips before sending the state to the client. However, note that the simulator σ_S has to arrange the order of these steps differently than described in the text above; first, it interacts with the server for all nodes and then with the client. This guarantees that the simulator has all the required information in the interaction with the client to work in the actual computation as well, and does not influence its correctness since operations on different parts of entangled states commute. The simulator is then the same as in Ref. [11], which means that the computational round also

does not introduce distinguishability. Hence we find with $\mathbf{QC}$ being a quantum and classical communication channel between the server and the client and $\mathbf{Q}$ just a quantum channel, $\pi_C^{StabPS} \mathbf{QC} = \pi_C^{HM15'} \mathbf{QC} \sigma_S = \pi_C^{HM15} \mathbf{QC} (\sigma'_S \circ \sigma_S)$.

Now consider a more general setting in which the client in a protocol $\pi^{g, RM}$ measures in RM any (to server and simulator unknown) generator g . We again use the simulator σ_S for which we already know the behavior if a node corresponds to a Z or X element in the g . Further, if the stabilizer element is $\mathbb{1}$, the state the server gets for this node is $\mathbb{1}/2$ as in the specifications in Section 3. However, if the stabilizer element is Y the situation is a bit more intricate since

$$\begin{aligned} & \text{Tr}_n \left(\left(Z_n^r |Y(0)_n\rangle \langle Y(0)_n| Z_n^r \otimes Z_{n'}^{\theta_n} \right) \frac{(|0_n 0_{n'}\rangle + |1_n 1_{n'}\rangle)(\langle 0_n 0_{n'}| + \langle 1_n 1_{n'}|)}{2} \left(\mathbb{1}_n \otimes (Z_{n'}^{\theta_n})^\dagger \right) \right) \\ &= Z_{n'}^{\theta_n} Z_{n'}^{r \oplus 1} |Y(0)_{n'}\rangle \langle Y(0)_{n'}| Z_{n'}^{r \oplus 1} Z_{n'}^{\theta_n}. \end{aligned} \quad (24)$$

Hence, the simulator introduces a bit-flip in the result. Since the simulator does not know the positions of Y elements in the stabilizer, it is not an option to use a different entangled state for these nodes. Nevertheless, two methods can be applied here. First, one can change the PS protocol to already respect this bit-flip. The correctness analysis in Section 3 would remain valid, since this essentially means to consider the transpose of the operator in the trace, leaving the trace and other elements in the protocol invariant – note that only the only $|Y(0)\rangle \langle Y(0)|$ is not symmetric in the analysis and all other operators are symmetric. Second, one might note that stabilizers of graph states have only even numbers of Y element. Therefore, the bit-flip does not change whether the client accepts or not and the offset θ_n guarantees that no distinguisher can tell apart whether there was a bit-flip in the outcome or not, since the outcomes themselves are not revealed but only their parity. Finally note that this analysis shows a reduction from the PS protocol $\pi^{g, PS}$ obtained from $\pi^{g, RM}$ by the procedure described in Section 3 to a RM protocol $\pi^{g, RM'}$, where the client measures g but respects in their decision random bit-flips introduced and announced by the server. As for the example, such bit-flips can be introduced (or canceled) by a simulator σ'_S showing eventually that the PS protocol is no less secure than its counterpart in RM, i.e., $\pi_C^{g, PS} \mathbf{QC} = \pi_C^{g, RM'} \mathbf{QC} \sigma_S = \pi_C^{g, RM} \mathbf{QC} (\sigma'_S \circ \sigma_S)$.